CodeClash

A live coding and quiz contest platform

OVERVIEW

CodeClash runs live coding and quiz contests. Participants reserve a limited number of seats, submit code that is judged in a sandbox, and watch a leaderboard that updates as verdicts come in. There are four roles, each with its own dashboard: participant, problem setter, organiser and admin. Unlike Prutor, which grades each student on their own work, CodeClash is built around users competing for shared, limited things: seats, judge time and rank. Past contests can be replayed in practice mode as a virtual contest, scored with the same rules and ranked against the original results. We do not reuse any Prutor code.

CONCURRENCY

Seats (main experiment). A seat is taken with one conditional update (taken < capacity) inside the transaction that creates it; if that fails, the user joins the waitlist. A withdrawal frees the seat and promotes the next waitlisted user in the same transaction. We will fire 400 sign-ups at 200 seats at once and check that nobody is double-booked and no contest is overfilled. 200 is roughly a department-level contest, and we will repeat the test with smaller capacities.

Judge slots. Each worker offers one slot per CPU core, so three 8-core workers give 24. This is a starting estimate that we will adjust once we measure real judging time. A free slot takes the next submission from a Redis queue (contest submissions before practice) and records a claim token and a short lease in Postgres, which it renews while running. If a worker stops renewing, its submission is handed to another slot with a new token; after three failures the organiser is alerted. Admins can drain or remove workers.

If a worker dies after running code but before saving the verdict, nothing was saved, so the submission is judged again. A verdict is saved only if the worker's token is still current, so a stalled worker that wakes up late cannot overwrite anything. Each submission can be scored once per judging, and scores are recalculated from verdicts rather than added, so a repeat can never double-count. We will test this by killing workers mid-run.

SCORING

ICPC-style ranking: first by problems solved, then by penalty, where penalty = minutes to each accepted solution + 20 minutes per earlier wrong attempt. Compile errors do not count as attempts. The first person to solve a problem gets a badge, decided by submission time rather than judging order. The leaderboard freezes for the final hour. Quiz answers score $1000 \times (1 - 0.5 \times t/T)$ for a correct answer in t seconds out of T, timed on the server.

AI AGENTS

Test Auditor generates extra tests and runs them in the sandbox. It can flag a weak problem by itself, but only the setter can add its tests or publish. **Integrity Analyst** compares submissions for similarity and writes a report. It cannot penalise anyone; the organiser decides. **Clarification Triage** answers simple rule questions privately and passes anything about problem content to the setter. These limits are enforced in the backend, not just in prompts: agents only have the tools listed here, each tool call goes through an API check tied to that agent's token, and the agents' database account cannot change seats, scores or submissions. We will show that blocked actions return an error even when the agent is told to try them.

SANDBOX AND SECURITY

All submitted code is treated as hostile. Each run uses nsjail with no network access, a read-only filesystem, memory, CPU and process limits, and a seccomp filter on system calls. Test data is never visible inside the sandbox. We will keep a set of malicious programs (fork bombs, memory hogs, infinite output, network and file-reading attempts) and run them in CI to confirm each one is stopped. Sandbox escape is our biggest security risk, and nobody on the team has built a sandbox before, so we start it in the first sprint. If nsjail proves too hard, we will fall back to a locked-down Docker container per submission.

STACK AND TOOLS

Next.js, NestJS, PostgreSQL, Redis, a Python judge worker and Claude for the agents, deployed on AWS with GitHub Actions. Tested with Jest/Vitest, pytest, Playwright and k6.

Jira: https://cs455-codeclash.atlassian.net/
GitHub: https://github.com/SrujanBhirud/CodeClash
The instructor and TAs (Shrikant, Ankit, Vansh) have been added to both.